

Q1.py X

Users > sanamuniralam > Desktop > Q1.py > parse_name

```
1  import socket
2  import struct
3  import random
4  import re
5
6  DNS_PORT = 53
7  QUERY_TIMEOUT = 5
8  MAX_UDP_SIZE = 4096
9
10 RCODE_MESSAGES = {
11     0: 'NOERROR',
12     1: 'FORMERR',
13     2: 'SERVFAIL',
14     3: 'NXDOMAIN',
15     4: 'NOTIMP',
16     5: 'REFUSED'
17 }
18
19
20 def is_valid_domain(domain):
21     if not domain or len(domain) > 253:
22         return False
23     pattern = r'^(?!-)[A-Za-z0-9-]{1,63}(?!-)(\.?(?!-)[A-Za-z0-9-]{1,63}(?!-))+$'
24     return re.match(pattern, domain) is not None
25
26
27 def encode_domain_name(domain):
28     parts = domain.strip('.').split('.')
29     encoded = b''
30     for part in parts:
31         encoded += struct.pack('B', len(part)) + part.encode('ascii')
32     encoded += b'\x00'
33     return encoded
34
35
36 def build_dns_query(domain, qtype=1, qclass=1):
37     transaction_id = random.randint(0, 65535)
38     flags = 0x0100
39     qdcount = 1
40     ancount = 0
41     nscount = 0
42     arcount = 0
43     header = struct.pack('>HHHHH', transaction_id, flags, qdcount, ancount, nscount, arcount)
44     question = encode_domain_name(domain) + struct.pack('>HH', qtype, qclass)
45     return transaction_id, header + question
46
```

```

46
47 # Parse a domain name from DNS response data
48 def parse_name(data, offset):
49     labels = []           # Initialize an empty list to hold the labels of the domain name
50     jumped = False        # Initialize a flag to indicate if a jump (pointer) has occurred in the name parsing
51     original_offset = offset # Store the original offset to return to after following a pointer
52     hops = 0              # Initialize a counter to track the number of jumps (pointers) followed to prevent infinite loops
53     while True:
54         if offset >= len(data): # Check if the offset has exceeded the length of the data to avoid index errors
55             break
56         length = data[offset]    # Read the length of the next label from the data at the current offset
57         if length == 0:          # Check for the end of the domain name (null label)
58             offset += 1
59             break
60         if (length & 0xC0) == 0xC0: # Check if the length indicates a pointer (the two highest bits are set)
61             if offset + 1 >= len(data): # Check if there is enough data for the pointer
62                 break
63             pointer = ((length & 0x3F) << 8) | data[offset + 1] # Calculate the pointer offset by combining the current length and the next byte
64             if not jumped: # If this is the first jump, store the original offset to return to after following the pointer
65                 original_offset = offset + 2 # Move the offset past the pointer bytes
66             offset = pointer # Set the offset to the pointer location to continue parsing the domain name from there
67             jumped = True    # Indicate that a jump has occurred to prevent returning to the original offset prematurely
68             hops += 1        # Increment the hop counter to track the number of jumps followed
69             if hops > 20:    # Prevent infinite loops by limiting the number of jumps (pointers) followed to 20
70                 break
71             continue
72         offset += 1
73         labels.append(data[offset:offset + length].decode('ascii', errors='replace')) # Decode the label from bytes to a string and append it to the labels list
74         offset += length
75     final_offset = original_offset if jumped else offset # Determine the final offset to return based on whether a jump occurred or not
76     return '.'.join(labels), final_offset
77
78

```

```

79 def parse_dns_response(data, expected_id):
80     if len(data) < 12:          # Check if the response is at least 12 bytes long (minimum DNS header size)
81         raise ValueError('Response too short to contain a valid DNS header')
82
83     resp_id, flags, qdcount, ancount, nscount, arcount = struct.unpack('>HHHHHH', data[:12])    # Unpack the DNS header
84
85     if resp_id != expected_id: # Check for transaction ID mismatch
86         raise ValueError('Transaction ID mismatch between query and response')
87
88     rcode = flags & 0xF        # Extract the response code from the flags
89
90     offset = 12                # Set the initial offset to the end of the header for parsing the question section
91     if qdcount < 1:
92         raise ValueError('Response contains no question section')
93
94     qname, offset = parse_name(data, offset)          # Parse the question name from the response data
95     if offset + 4 > len(data):
96         raise ValueError('Malformed question section')
97     qtype, qclass = struct.unpack('>HH', data[offset:offset + 4]) # Unpack the question type and class from the response data
98     offset += 4                                         # Move the offset past the question section
99
100    answers = []
101    for _ in range(ancount):                             # Loop through each answer record in the response
102        aname, offset = parse_name(data, offset)          # Parse the answer name from the response data
103        if offset + 10 > len(data):                       # Check if there are enough bytes left for the answer record header
104            raise ValueError('Malformed answer record')
105        atype, aclass, attl, rdlength = struct.unpack('>HHIH', data[offset:offset + 10])    # Unpack the answer type, class, TTL, and RDLENGTH from the response data
106        offset += 10                                     # Move the offset past the answer record header
107        if offset + rdlength > len(data):                 # Check if there are enough bytes left for the RDATA based on the RDLENGTH
108            raise ValueError('Malformed answer RDATA')
109        rdata = data[offset:offset + rdlength]            # Extract the RDATA from the response data based on the RDLENGTH
110        offset += rdlength
111
112        ip_address = None                                # Initialize the IP address. Will store resolved IP if the answer is of type A (IPv4 address)
113        if atype == 1 and aclass == 1 and rdlength == 4: # Check if the answer is of type A and class IN (Internet) with a valid RDLENGTH
114            ip_address = socket.inet_ntoa(rdata)         # Convert the 4-byte RDATA to a human-readable IPv4 address using socket.inet_ntoa
115
116        answers.append({                                  # Store the parsed answer record details in a dictionary and append it to the answers list
117            'name': aname,
118            'type': atype,
119            'class': aclass,
120            'ttl': attl,
121            'ip': ip_address
122        })
123
124    return {                                              # Return a dictionary containing the parsed DNS response details
125        'id': resp_id,
126        'rcode': rcode,
127        'qname': qname,
128        'qtype': qtype,
129        'ancount': ancount,
130        'answers': answers
131    }
132

```

```

133 def resolve_domain(domain, dns_server):
134     transaction_id, query = build_dns_query(domain)
135     sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
136     sock.settimeout(QUERY_TIMEOUT)
137     try:
138         sock.sendto(query, (dns_server, DNS_PORT))
139         data, _ = sock.recvfrom(MAX_UDP_SIZE)
140     except socket.timeout:
141         print('Request timed out. No response received from DNS server.')
142         return
143     except socket.error as exc:
144         print(f'Socket error occurred: {exc}')
145         return
146     finally:
147         sock.close()
148
149     try:
150         result = parse_dns_response(data, transaction_id)
151     except ValueError as exc:
152         print(f'Malformed or unexpected DNS response: {exc}')
153         return
154
155     print('-' * 55)
156     print(f'Transaction ID      : {result["id"]}')
157     print(f'Query Name              : {result["qname"]}')
158     print(f'Query Type               : {"A" if result["qtype"] == 1 else result["qtype"]}')
159     print(f'Response Status          : {RCODE_MESSAGES.get(result["rcode"], "UNKNOWN")}')
160     print(f'Answer Count             : {result["ancount"]}')
161
162     if result['rcode'] == 3:
163         print('Domain does not exist (NXDOMAIN).')
164     elif result['ancount'] == 0:
165         print('No answer records returned for this query.')
166     else:
167         for index, answer in enumerate(result['answers'], start=1):
168             if answer['ip']:
169                 print(f'Answer {index}          : {result["qname"]} -> {answer["ip"]} (TTL: {answer["ttl"]}s)')
170             else:
171                 print(f'Answer {index}          : Non-A record (Type {answer["type"]}), TTL: {answer["ttl"]}s')
172     print('-' * 55)
173
174 def main():
175     dns_server = input('Enter DNS server IP address (e.g., 8.8.8.8): ').strip()
176     while True:
177         domain = input('\nEnter domain name to resolve (or type "exit" to quit): ').strip()
178         if domain.lower() == 'exit':
179             break
180         if not is_valid_domain(domain):
181             print('Invalid domain name format. Please try again.')
182             continue
183         resolve_domain(domain, dns_server)
184
185 if __name__ == '__main__':
186     main()

```

```
● sanamuniralam@Sanas-MacBook-Air Desktop % python3 Q1.py
Enter DNS server IP address (e.g., 8.8.8.8): 8.8.8.8

Enter domain name to resolve (or type "exit" to quit): google.com
-----
Transaction ID   : 8882
Query Name      : google.com
Query Type      : A
Response Status  : NOERROR
Answer Count    : 1
Answer 1        : google.com -> 142.250.186.14 (TTL: 8s)
-----

Enter domain name to resolve (or type "exit" to quit): wikipedia.org
-----
Transaction ID   : 53282
Query Name      : wikipedia.org
Query Type      : A
Response Status  : NOERROR
Answer Count    : 1
Answer 1        : wikipedia.org -> 103.102.166.224 (TTL: 81s)
-----

Enter domain name to resolve (or type "exit" to quit): github.com
-----
Transaction ID   : 19251
Query Name      : github.com
Query Type      : A
Response Status  : NOERROR
Answer Count    : 1
Answer 1        : github.com -> 20.207.73.82 (TTL: 1s)
-----

Enter domain name to resolve (or type "exit" to quit): this-domain-definitely-does-not-exist-928374.com
-----
Transaction ID   : 17746
Query Name      : this-domain-definitely-does-not-exist-928374.com
Query Type      : A
Response Status  : NXDOMAIN
Answer Count    : 0
Domain does not exist (NXDOMAIN).
-----

Enter domain name to resolve (or type "exit" to quit): exit
○ sanamuniralam@Sanas-MacBook-Air Desktop %
```

Computer Network [Assignment - 1]

Q1) Part a

$$\begin{aligned}
 \text{Total Time} &= T_{\text{ROOT}} + T_{\text{TLD}} + T_{\text{AUTHORITATIVE}} \\
 &= 60 + 50 + 40 \\
 &= 150 \text{ ms}
 \end{aligned}$$

Part b

$$\begin{aligned}
 \text{Time Saved} &= T_{\text{Total}} - T_{\text{AUTHORITATIVE}} \\
 &= 150 - 40 \\
 &= 110 \text{ ms}
 \end{aligned}$$

Q2) Part a

$$\text{TTL in min} = \frac{300}{60} = 5 \text{ minutes}$$

$$\text{Cached Time} = 10:00:00 \text{ am}$$

$$\text{Query time} : 10:03:00 \text{ AM}$$

$$\text{Time still remaining} = 5 - 3 = 2 \text{ min}$$

Since we still have time (2 min) left for expiry. The ~~query~~ cached entry is still valid.

$$\begin{aligned}
 \text{Time Saved} &= T_{\text{Full Resolution}} - T_{\text{Cache Lookup}} \\
 &= 150 - 2 \\
 &= 148 \text{ ms} \quad \text{saved}
 \end{aligned}$$

Part b

$$\text{Query time in second (7 min)} = 7 \times 60 = 420 \text{ sec}$$

As $420 > 300$ the cached entry has expired. Therefore entry must be discarded for a fresh full resolution is required.

Q3) Link 1

$$\begin{aligned}
 d_{\text{TRANS}} &= L/R = (8 \times 12 \times 10^6) / (20 \times 10^6) = 4.8 \text{ seconds} \\
 &= \cancel{(8 \times 10^6)} / \cancel{(20 \times 10^6)}
 \end{aligned}$$

Link 2

$$d_{\text{TRANS}} = L/R = (8 \times 12 \times 10^6) / (100 \times 10^6) = 0.96 \text{ seconds}$$

$$\text{Total Transmission Delay} = 4.8 + 0.96 = 5.76 \text{ seconds}$$

Date: _____

Q4) Link 1	Link 2	Link 3
$d_{\text{TRANS}} = \frac{8000}{40 \times 10^6}$	$d_{\text{TRANS}} = \frac{8000}{80 \times 10^6}$	$d_{\text{TRANS}} = \frac{8000}{500 \times 10^6}$
$= 0.2 \text{ ms}$	$= 0.1 \text{ ms}$	$= 0.016 \text{ ms}$
$d_{\text{PROP}} = 20 \text{ ms}$	$d_{\text{PROP}} = 30 \text{ ms}$	$d_{\text{PROP}} = 5 \text{ ms}$
$d_{\text{TOTAL}} = 20 + 0.2$	$d_{\text{TOTAL}} = 30 + 0.1$	$d_{\text{TOTAL}} = 5 + 0.016$
$= 20.2 \text{ ms}$	$= 30.1 \text{ ms}$	5.016 ms

$$\text{Total End-End Delay} = d_{\text{TOTAL LINK 1}} + d_{\text{TOTAL LINK 2}} + d_{\text{TOTAL LINK 3}}$$

$$= 20.2 + 30.1 + 5.016$$

$$= 55.316 \text{ ms}$$

Q5) Part a

$$\begin{aligned} \text{Total Traffic} &= \text{Obj size} * \text{Req per sec} \\ &= 5000 * (40 \times 10^3) \\ &= 2000 \text{ Mb/hr} \end{aligned}$$

Part b

$$\text{Cache Miss} = 1 - 75\% = 25\% \text{ or } 0.25$$

$$\text{Request Reaching Origin} = 5000 \times 0.25 = 1250 \text{ request}$$

$$\begin{aligned} \text{Total Traffic} &= 1250 * (40 \times 10^3) \\ &= 500 \text{ Mb/hr} \end{aligned}$$

Part c

$$\text{Amount of data traffic Saved} = \text{Traffic}_{\text{original}} - \text{Traffic}_{\text{after cache}}$$

$$= 2000 - 500$$

$$= 1500 \text{ Mb/hr saved}$$